CS6013 - Modern Compilers: Theory and Practise

Interprocedural Analysis

V. Krishna Nandivada

IIT Madras

Opening remarks

What have we done so far?

- Compiler overview.
- Scanning and parsing.
- JavaCC, visitors and JTB
- Semantic Analysis - specification, execution, attribute grammars.
- Type checking, Intermediate Representation, Intermediate code generation.
- Data flow analysis, intra-procedural constant propagation.
- Control flow analysis, interval analysis, structural analysis.

Announcement:

- Assignment 1: Due today.

Today:

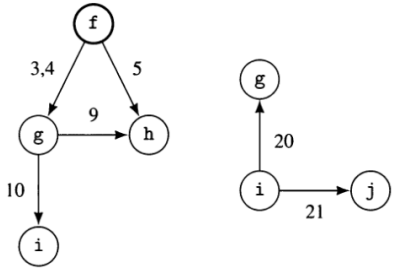
- Inter-procedural analysis.

Interprocedural CFA - Call graph

- Inter-procedural CFA constructs a static Call graph
  - A directed multigraph.
- Given a program P, consisting of procedures p_1, $p_2 \ldots p_n$, the call grpah $G = \langle N, S, E, r \rangle$
- N is the set of procedures.
- S is the set of call sites labels (e.g. line numbers in TAC).
- $E \subseteq N \times S \times N$: An edge from ($p_1$, s, p_2) indicates a call from p_1 to p_2 at site s.

Example call graph

```
procedure f( )
begin
   call g( )
   call g( )
   call h( )
end   || f
procedure g( )
begin
   call h( )
   call i( )
end   || g
procedure h( )
begin
end   || h
procedure i( )
   procedure j( )
   begin
   end   || j
begin
   call g( )
   call j( )
end   || i
```



Constructing the call graph

```
LabeledEdge = Procedure × integer × Procedure

procedure Build_Call_Graph(P,r,N,E,numinsts)
   P: in set of Procedure
   r: in Procedure
   N: out set of Procedure
   E: out set of LabeledEdge
   numinsts: in Procedure ⟶ integer
begin
   i: integer
   p, q: Procedure
   OldN := ∅: set of Procedure
   N := {r}
   E := ∅
   while OldN ≠ N do
      p := ◆(N - OldN)
      OldN := N
      for i := 1 to numinsts(p) do
         for each q ∈ callset(p,i) do
            N ∪= {q}
            E ∪= {⟨p,i,q⟩}
         od
      od
   od
end    || Build_Call_Graph
```

Challenges

- Overloaded functions and inheritance. (CHA - On the Board)
- Separate compilation – we would not know the complete call graph; wait till the whole program is available.
- Function pointers. Read yourself.

Interprocedural constant propagation

Two flavors of inter-procedural constant propagation.

- Context insensitive (call site independent) constant propagation.
  - For each procedure in a program identify the subset of its parameters, such that each of the parameter will get a constant value, in every invocation.
  - The return value may be constant for every invocation or none.
- Context sensitive (call site dependent) constant propagation:
  - for each particular procedure called from each particular site, the subset of parameters that have the same constant value each time the procedure is called at that site.
  - For each call site, the return value may be constant or not.

Interprocedural constant propagation overview

```
Function constantProp()
begin
    worklist = {root};
    while worklist is not empty do
        p = worklist.dequeue();
        // Intra CP as before.  Two Special cases
        Call-sites
            foreach callsite s in p do
                compute the actuals of s using the formals of p;
                Say the function being called at s is q;
                Compute the meet of the current values for the formals of q and the
                actuals at s;
                if constant values of q has changed then
                    add q to the worklist;
        Return-statements
            v = compute the meet of all the return values of p;
            Set the return value of p to v;
            foreach call function q that calls p do
                add q to the worklist
```

Initialization and modification to to CP algorithm

Initialization

- The return value of each function is initialized to $\top$.
- The constant value of each formal argument is initialized to $\top$.

Modification to the CP

- Constant value of a function call is given by the constant-return value of the function.
- If the statement is of the form $a = foo(\cdots)$, set the constant value of a to that of the function.

Function Inlining

(On Board)

Closing remarks

To read

- Muchnick - Ch 19.1, 19.3.